Cloud-Native Media Privacy Platform - Nền tảng ẩn danh hóa thông minh trên nền tảng Kubernetes

1. Bối cảnh

- **Sự bùng nổ của nội dung đa phương tiện và dữ liệu công cộng:** Trong kỷ nguyên số, việc quay phim, chụp ảnh và chia sẻ video ngắn (Vlog, TikTok, YouTube Shorts, Facebook Reels) diễn ra hàng ngày ở khắp mọi nơi. Cùng với đó là sự phổ cập của camera hành trình (dashcam) trên ô tô, xe máy và hệ thống camera giám sát đô thị. Vô tình, một lượng khổng lồ hình ảnh khuôn mặt của người qua đường, người tham gia sự kiện và biển số phương tiện cá nhân bị ghi lại và lan truyền công khai trên Internet.

- **Nhu cầu của người dùng:**
    - **Giảm chi phí hậu kỳ:** Nhà sáng tạo nội dung, nhiếp ảnh gia và đội ngũ truyền thông phải xử lý lượng lớn ảnh/video trước khi xuất bản. Tự động phát hiện và che khuôn mặt, biển số giúp giảm thời gian chỉnh sửa thủ công, tăng tốc độ bàn giao nội dung.
    - **Tạo phiên bản nội dung phù hợp để công khai:** Trường học, sự kiện và người dùng camera hành trình có thể cần chia sẻ tư liệu nhưng không cần tiết lộ danh tính của tất cả người xuất hiện. Nền tảng tạo bản sao đã che để công khai.
    - **Chia sẻ dữ liệu cho bên thứ ba có kiểm soát:** Các đơn vị vận hành camera, doanh nghiệp và nhóm phát triển AI sử dụng dữ liệu hình ảnh cho phân tích hoặc thử nghiệm. Tự động che các thông tin nhận dạng không cần thiết giúp giảm dữ liệu cá nhân được chuyển giao mà vẫn bảo toàn phần nội dung phục vụ công việc.
    - **Bảo vệ người không liên quan trong nội dung công khai:** Khi đăng video phản ánh giao thông, sự cố tại cửa hàng hoặc hoạt động nơi công cộng, người đăng thường quan tâm đến sự việc chứ không cần công khai danh tính của mọi người trong khung hình. Che tự

động giúp giữ trọng tâm nội dung trong khi giảm việc phát tán thông tin nhận dạng ngoài mục đích chính.

- **Sức ép pháp lý về quyền riêng tư và dữ liệu cá nhân:**
    - **Tại Việt Nam:** Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 và Nghị định 356/2025/NĐ-CP có hiệu lực từ ngày 01/01/2026, đặt ra khuôn khổ pháp lý hiện hành về bảo vệ dữ liệu cá nhân. Hình ảnh khuôn mặt và biển số xe có thể là dữ liệu cá nhân khi gắn với hoặc giúp xác định một cá nhân. Việc thu thập, sử dụng và công khai các dữ liệu này cần được xem xét theo mục đích xử lý, căn cứ pháp lý và các quy định áp dụng; đương nhiên, không phải mọi trường hợp ghi hình tại nơi công cộng đều mặc nhiên bị cấm nhưng rủi ro pháp lý có tồn tại.
    - **Trên thế giới:** Quy định chung về bảo vệ dữ liệu của Liên minh Châu Âu (GDPR) đặt ra các yêu cầu về tính hợp pháp, minh bạch, giới hạn mục đích và tối thiểu hóa dữ liệu trong hoạt động xử lý dữ liệu cá nhân của các tổ chức, doanh nghiệp. Việc không tuân thủ có thể dẫn đến các chế tài, tùy tính chất và mức độ vi phạm.

Các phương pháp hiện có và hạn chế

Phương pháp hiện có	Cách thức hoạt động	Ưu điểm	Hạn chế
1. Chỉnh sửa thủ công *(Adobe Premiere, DaVinci Resolve, CapCut)*	Người dựng phim tự tạo mask, chấm keyframe bằng tay qua từng giây video.	Độ chính xác cao theo ý muốn người dựng.	• Tốn nhiều thời gian và công sức (1 phút video có thể mất 30-60 phút cắt ghép). • Không thể tự động hóa cho hàng loạt video. • 20-30USD/tháng (CapCut và Adobe Premiere), 300USD để mua một lần (DaVinci)

Phương pháp hiện có	Cách thức hoạt động	Ưu điểm	Hạn chế
2. Công cụ Auto-blur phổ thông *(YouTube Studio Blur)*	Dùng thuật toán Haar Cascade hoặc model có sẵn để che mọi khuôn mặt tìm thấy.	Tự động hóa việc làm mờ, nhanh hơn làm tay. Miễn phí có giới hạn (dành cho video chưa edit lần nào, dưới 100k views).	• Xử lý và cấu hình thủ công từng video một, thiếu thốn về khả năng xử lý theo batch • Không có report về kết quả, độ tin cậy, phải rà soát thủ công • Không có che biển số
3. Dịch vụ AI Cloud thương mại *(Amazon Rekognition, Google Vision API)*	Gọi API trả tiền theo lượt.	Hạ tầng mạnh, độ chính xác nhận diện tổng quát tốt. Có free credit tuỳ dịch vụ	• Free credit không bao gồm lưu trữ, truyền dữ liệu, xử lý ảnh/video • Cần tự nối pipeline nhận diện → làm mờ (không có sẵn tính năng làm mờ) • AWS Object Detection: 1 USD/1.000 ảnh → Video 10 phút = 36 USD
Công cụ mã nguồn mở như `deface`, `video-privacy-blur`	Người dùng tự clone về và host AI model để sử dụng	Miễn phí, chạy local, không cần gọi API trả phí cho mỗi ảnh	• Tập trung vào xử lý file • Không hỗ trợ batching, nếu có thì sử dụng CLI • Thiếu thốn về khả năng quản lý, chỉ là một công cụ xử lý thuần

2. Mức độ giải quyết

Hiện nay, bài toán ẩn danh hóa hình ảnh và video có thể được giải quyết bằng các công cụ mã nguồn mở chạy cục bộ hoặc các ứng dụng xử lý media có sẵn.

Các giải pháp này đã cung cấp những chức năng cơ bản như phát hiện khuôn mặt, phát hiện biển số và làm mờ tự động. Vì vậy, đề tài không đặt mục tiêu xây dựng lại một thuật toán nhận diện đã có, mà tập trung giải quyết bài toán tổ chức và vận hành quá trình xử lý media ở quy mô nhỏ-lớn để có thể cung cấp một nền tảng tiện ích, đáp ứng nhu cầu của người dùng.

Kết quả của nhóm sẽ là một nền tảng hoạt động hoàn toàn tự động, mã nguồn mở, có thể self-host, dành riêng cho tác vụ bảo vệ quyền riêng tư, xử lý các tập, bộ media với khả năng mở rộng và tích hợp trong tương lai.

→ Có script tự động apply manifests hoặc tạo cụm k3s, kind. Nếu không muốn Kubernetes có thể sử dụng Docker Compose, tuy nhiên sẽ mất đi khả năng tự scale của KEDA

3. Mục tiêu chức năng

Luồng trải nghiệm người dùng:

- Tải lên: Tải tập ảnh hoặc video lên thư viện media.
- Cấu hình: Chọn loại dữ liệu cần che và phong cách che.
- Xử lý: Job đi qua hàng đợi để chờ xử lý.
- Kiểm chứng: Nhận bản đã ẩn danh, trạng thái job và báo cáo.

Nền tảng được phát triển với 4 nhóm chức năng chính:

3.1. Ẩn danh hóa tự động

- **Xử lý Ảnh tĩnh:** Hỗ trợ các định dạng phổ biến (JPG, PNG, WEBP), xử lý tức thì, cung cấp thanh trượt so sánh trực quan *Before / After*.
- **Xử lý Video ngắn:** Tự động bóc tách từng khung hình, áp dụng ẩn danh hóa, tích hợp thuật toán theo dõi đối tượng (ByteTrack), vết mờ di chuyển mượt mà, không bị hiện tượng nhấp nháy (flickering). Sử dụng **FFmpeg** để bảo toàn nguyên vẹn 100% âm thanh gốc của video. Đầu ra có chất lượng theo cấu hình của người dùng (vd: 75%-90%, 720p hoặc giữu nguyên...)
- Xoá dữ liệu định kì hoặc khi người dùng yêu cầu

Notes ko cần cho vào slide:

- ByteTrack – theo dõi đối tượng: YOLO phát hiện đối tượng độc lập ở từng frame, nên bounding box có thể thay đổi vị trí hoặc biến mất trong một vài frame, gây hiện tượng nhấp nháy ở vùng làm mờ. ByteTrack liên kết các detection giữa nhiều frame để duy trì ID theo dõi của đối tượng, giúp hệ thống xác định các detection thuộc cùng một đối tượng và làm vùng làm mờ ổn định hơn. Chọn ByteTrack vì đây là thuật toán tracking phổ biến, có thể kết hợp với detection từ YOLO mà không cần huấn luyện một mô hình nhận diện mới. Tuy nhiên, ByteTrack không đảm bảo loại bỏ hoàn toàn flickering, nhất là khi bị che khuất hoặc detector bỏ sót đối tượng.

- FFmpeg – xử lý và ghép video: FFmpeg hỗ trợ giải mã video thành frame, xử lý luồng âm thanh và mã hóa/ghép lại thành video đầu ra. Nhóm chọn FFmpeg vì hỗ trợ nhiều codec và định dạng media, phù hợp để xây dựng pipeline xử lý tự động. Có thể giữ nguyên luồng âm thanh bằng cách sao chép audio stream thay vì mã hóa lại, nhưng cần kiểm tra codec và định dạng đầu ra có tương thích hay không.

- **Đa dạng phong cách che:**
    - *Gaussian Blur:* Làm mờ mềm mại, tự nhiên (phù hợp vlog đời sống).
    - *Pixelate (Mosaic 8-bit):* Phong cách phóng sự truyền hình, tin tức.
    - *Blackout Tape:* Băng che đen tuyệt đối, phong cách hồ sơ pháp lý/tài liệu mật.
- **Xử lý hàng loạt:** Tiếp nhận và phân phối nhiều tác vụ xử lý ảnh/video, theo dõi trạng thái, hỗ trợ thử lại khi xảy ra lỗi và hạn chế việc phải xử lý lại toàn bộ quy trình.
- **Khả năng cấu hình:** Người dùng có thể tuỳ chọn cấu hình tác vụ ẩn danh theo từng job. Một job chỉ che khuôn mặt; job khác che cả khuôn mặt và biển số; job khác nữa yêu cầu che mạnh hơn, tạo báo cáo về job đã xử lý. UI sẽ hiện trước kết quả và cho phép người dùng cấu hình độ mờ, độ lớn của các hạt,...

3.2. Ẩn danh có chọn lọc

- **Lọc theo khuôn mặt:** Người dùng chỉ cần tải lên 1 hoặc nhiều ảnh chân dung mẫu (Selfie). Hệ thống sử dụng mô hình trích xuất đặc trưng khuôn mặt (Face Embeddings) để so khớp: **Giữ nguyên 100% độ nét khuôn mặt mẫu**, nhưng **tự động che mờ tất cả người lạ đi đường**.
- **Lọc theo biển số xe:** Cho phép đăng ký biển số xe cá nhân để hệ thống không làm mờ biển số của mình mà chỉ làm mờ các xe xung quanh.
- **Dữ liệu linh hoạt**: Người dùng tạo các profile và lọc theo một hoặc nhiều profile cho từng batch. Có thể upload ảnh để lọc mà không cần profile.

3.3. Bản địa hóa giao thông Việt Nam

Mô hình được tinh chỉnh đặc thù để nhận diện chính xác:

- Biển số xe máy Việt Nam (dạng vuông, 2 dòng ký tự) ở các góc chụp nghiêng, ngược sáng.
- Biển số ô tô, biển số xe kinh doanh (biển vàng), biển xe công vụ.

Đã có các dự án mã nguồn mở sử dụng YOLO và OCR để phát hiện, xử lý biển số Việt Nam. Tuy nhiên về hiệu năng trong các video thực tế, bao gồm biển nhỏ, nghiêng, rung, mờ hoặc bị che khuất, sự hoạt động ổn định trong điều kiện mật độ phương tiện đông đúc đặc trưng của đô thị Việt Nam cần kiểm thử bằng dữ liệu và tinh chỉnh.

3.4. Báo cáo và thống kê kết quả

Hệ thống đóng vai trò như một công cụ kiểm toán an toàn dữ liệu:

- Phân tích và thống kê số lượng khuôn mặt, số lượng biển số xe xuất hiện trong media.
- Thống kê các ảnh có tỉ lệ confidence thấp để người dùng có thể tự kiểm tra lại kết quả một cách chọn lọc
- Cho phép xuất Audit Report dạng JSON/PDF

---

5. Giải pháp của nhóm

A. Về mặt Trí tuệ nhân tạo:

- **Dataset & Fine-tuning:** Không dùng model có sẵn đơn thuần. Nhóm thu thập, tiền xử lý và gán nhãn tập dữ liệu kết hợp (WIDER FACE và Vietnamese License Plate Dataset) để huấn luyện mô hình **YOLOv8n** chuyên biệt.
- **Mục tiêu độ chính xác:** Mục tiêu ban đầu đạt **mAP@50 ≥ 85%** trên tập kiểm thử cho cả 2 nhãn khuôn mặt và biển số xe Việt Nam. Đặc biệt ưu tiên Recall cao (tránh bỏ sót người lạ)
- **Tính năng Whitelist:** Ứng dụng mô hình trích xuất đặc trưng khuôn mặt (Cosine Similarity trên không gian embedding) để lọc danh tính chủ thể với độ chính xác nhận diện người quen ≥ 90%.
- **Tối ưu hóa suy luận:** Chuyển đổi mô hình sang định dạng ONNX, sử dụng OpenVINO INT8 giúp giảm mức tiêu thụ RAM và tăng tốc khi chạy trên môi trường server chỉ có CPU. Mục tiêu là cải thiện tốc độ 2-3 lần.

(ko cần trong slides. note để hiểu / thầy hỏi thì trả lời)

- Tại sao chọn YOLOv8n?

- YOLOv8n là phiên bản nhỏ gọn của YOLOv8, có số lượng tham số và chi phí tính toán thấp hơn các phiên bản lớn, phù hợp với môi trường server CPU có tài nguyên hạn chế.
- Hỗ trợ phát hiện khuôn mặt và biển số trong cùng một mô hình bằng cách huấn luyện trên dataset có hai nhãn tương ứng, giúp đơn giản hóa việc triển khai và giảm chi phí vận hành.
- Phù hợp với mục tiêu triển khai nhiều worker xử lý ảnh/video đồng thời. Tuy nhiên, cần kiểm thử thực tế để xác nhận tốc độ và độ chính xác trên dữ liệu của đề tài.

- mAP@50 là gì? Tại sao chọn ngưỡng ≥ 85%?

- mAP@50 là mean Average Precision tại ngưỡng IoU (Intersection over Union) = 0,5. IoU đo mức độ chồng lấp giữa vùng đối tượng mà mô hình dự đoán và vùng được gán nhãn; dự đoán được xem là đúng khi IoU đạt ít

nhất 0,5, đồng thời xét đến precision và recall trên các ngưỡng confidence.

- Precision (độ chính xác của các dự đoán dương tính): Trong số các đối tượng AI dự đoán là khuôn mặt hoặc biển số xe, có bao nhiêu đối tượng thực sự là khuôn mặt hoặc biển số xe?
- Recall (độ bao phủ) là tỷ lệ khuôn mặt và biển số xe thực tế được AI phát hiện thành công trên tổng số khuôn mặt và biển số xe có trong ảnh. Trong tổng số khuôn mặt và biển số xe thực tế, AI phát hiện được bao nhiêu?
    - $Recall = \frac{Số đối tượng phát hiện đúng}{Tổng số đối tượng thực tế}$
- Chọn mAP@50 ≥ 85% làm mục tiêu đánh giá ban đầu để đặt ra tiêu chí định lượng rõ ràng cho khả năng phát hiện khuôn mặt và biển số. Đây là ngưỡng nhóm đề xuất, không phải tiêu chuẩn bảo đảm chất lượng ẩn danh hóa.
- Cần đánh giá thêm precision, recall và kết quả riêng trên từng lớp. Đặc biệt, bỏ sót một khuôn mặt hoặc biển số có thể khiến thông tin nhận dạng vẫn bị lộ; mAP cao không đồng nghĩa với việc mọi đối tượng đều được che đúng.

- Tính năng Whitelist: Tại sao dùng Face Embedding và Cosine Similarity?

- Face embedding chuyển khuôn mặt thành một vector đặc trưng số; cosine similarity đo mức độ tương đồng giữa vector của khuôn mặt trong ảnh và vector danh tính đã đăng ký.
- Cách này cho phép nhận diện chủ thể cần giữ nguyên mà không phải so sánh ảnh theo từng pixel, phù hợp hơn khi khuôn mặt thay đổi góc nhìn hoặc điều kiện ánh sáng.
- Mục tiêu nhận diện người quen ≥ 90% cần được kiểm thử trên dữ liệu riêng, với ngưỡng tương đồng được hiệu chỉnh. Phải ưu tiên tránh nhận nhầm người lạ thành người trong whitelist, vì lỗi này có thể khiến khuôn mặt cần che bị bỏ sót.

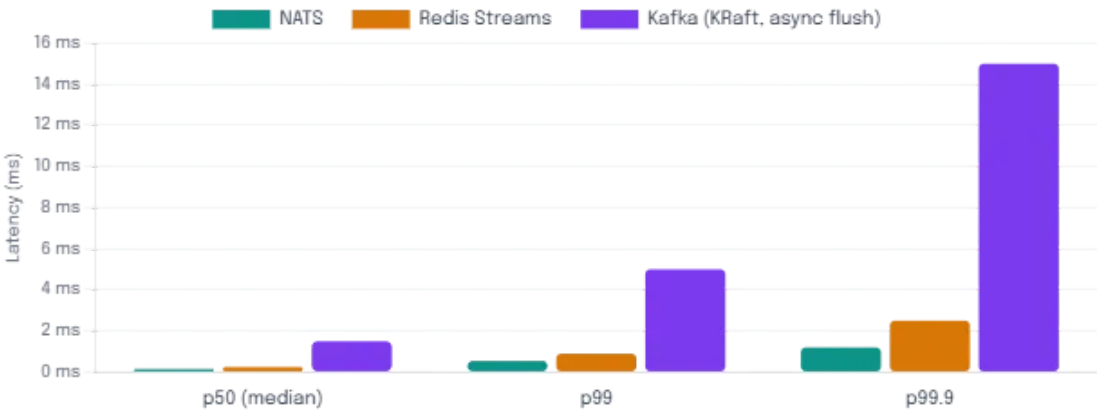
- ONNX Runtime / OpenVINO INT8 là gì? Tại sao chọn?

- Model format – ONNX: Là định dạng trao đổi mô hình, giúp tách biệt framework huấn luyện với môi trường triển khai. Nhóm chọn ONNX nếu muốn dễ thử nghiệm nhiều runtime khác nhau mà không phải huấn luyện lại YOLOv8n. Đổi lại, cần kiểm tra tính tương thích khi chuyển đổi mô hình.
- Model runtime – OpenVINO Runtime: Là môi trường thực thi mô hình, được thiết kế để tối ưu suy luận trên phần cứng Intel (hạ tầng của nhóm), đặc biệt là CPU. OpenVINO có thể trực tiếp nhập mô hình ONNX.
- Inference optimization – INT8 quantization: Là kỹ thuật giảm độ chính xác số học của trọng số và phép tính xuống mức 8-bit để giảm bộ nhớ và có khả năng tăng tốc suy luận. Đổi lại, mô hình có thể phát hiện sai hoặc bỏ sót nhiều đối tượng hơn. Với bài toán ẩn danh hóa, đây là rủi ro đáng kể vì bỏ sót khuôn mặt hoặc biển số có thể làm lộ thông tin cá nhân. Do đó, nhóm chỉ sử dụng INT8 nếu thử nghiệm chứng minh hiệu năng được cải thiện mà độ chính xác vẫn đáp ứng yêu cầu.
- Quyết định triển khai: Sử dụng YOLOv8n kết hợp OpenVINO Runtime trên CPU làm phương án chính; benchmark với mô hình ban đầu. INT8 là bước tối ưu bổ sung có điều kiện, không mặc định áp dụng. Đánh giá dựa trên throughput, latency, RAM và các chỉ số phát hiện trước/sau tối ưu hóa.

B. Về mặt Hệ thống & Hạ tầng Cloud-Native):

- **Kiến trúc hướng sự kiện:** Sử dụng **NATS JetStream** để phân tách hoàn toàn tầng Web tiếp nhận và tầng Worker AI xử lý, loại bỏ tình trạng nghẽn kết nối khi có nhiều request đồng thời. NATS JetStream hoàn toàn open-source (không như Redis) và nhẹ, đơn giản hơn Kafka rất nhiều.

Bảng metrics phân phối độ trễ: p50, p99, p99.9 tại 3,000 req/s (1 KB payload): pN = a nghĩa là N% số lượng request có độ trễ nhỏ hơn a



Approximate values drawn from published benchmarks — Brave New Geek & Synadia (2024). Lower is better.

- **Tự động co giãn theo tải thực tế:**
    - Định nghĩa *ScaledObject* lắng nghe chỉ số độ trễ/số lượng tin nhắn chờ trong hàng đợi NATS. Tốt hơn HPA (Horizontal Pod Autoscaler) thông thường chỉ dựa vào metrics (RAM, CPU) của cụm.
    - Tự động co giãn số lượng Worker Pods: Thử nghiệm với các chiến thuật scale worker, phân phối event stream khác nhau. Ví dụ: **Scale-out từ 1 pod lên 4–8 pods** khi có video dài hoặc nhiều tác vụ gửi đến, và **scale-in về 0-1 pod** khi hết việc để tiết kiệm tài nguyên.
- **Đo đạc & Thực nghiệm khoa học:** Có số liệu benchmark cụ thể so sánh với các tool opensource, các ứng dụng có sẵn:
    - Thời gian xử lý: thời gian xử lý trên cùng tập ảnh/video
    - Hiệu năng suy luận: độ chính xác, thông lượng, độ trễ, tài nguyên của hệ thống khi tăng số worker

⇒ Tại sao chọn kiến trúc Cloud-Native (ko cần trong slides, thầy hỏi thì trả lời):

- **Tự động mở rộng tài nguyên:** Điều chỉnh số lượng worker theo nhu cầu xử lý, giảm tài nguyên nhàn rỗi khi không có tác vụ và tăng năng lực xử lý khi hàng đợi tăng.
- **Chủ động về triển khai và luồng dữ liệu:** Có thể triển khai trong hạ tầng riêng nhanh chóng, hỗ trợ các đơn vị cần kiểm soát nơi lưu trữ và quá trình xử lý media.

- **Đánh giá được hiệu quả hệ thống:** Có thể đo thông lượng, thời gian xử lý, khả năng mở rộng, mức sử dụng tài nguyên và khả năng phục hồi khi worker gặp lỗi.

⇒ Tại sao chọn event-driven thay vì queue:

- **Rủi ro mất dữ liệu khi Worker Crash:** Các tác vụ AI xử lý ảnh rất nặng và rủi ro pod bị Kubernetes "kill" do tràn RAM (OOMKilled) là rất cao. Normal queues thường xóa message ngay khi worker vừa nhận. Nếu dùng NATS JetStream, hệ thống có cơ chế At-Least-Once Delivery: nếu worker chết giữa chừng mà chưa gửi tín hiệu hoàn tất (Ack), message sẽ tự động được đưa lại vào hàng đợi để worker khác xử lý tiếp.
- **Không thể tự động Scale theo độ dài hàng đợi (KEDA Integration):** Normal queues không giao tiếp trực tiếp với bộ điều khiển tài nguyên của Kubernetes (HPA). Khi có 1,000 tấm ảnh được tải lên cùng lúc, các AI worker thông thường sẽ bị quá tải và hàng đợi cứ thế kéo dài. Việc sử dụng KEDA kết nối với NATS cho phép Kubernetes "nhìn" thấy số lượng ảnh đang chờ để tự động đẻ thêm Pod (Scale Out) xử lý, và tự động thu gọn về 0/1 khi hết việc để tiết kiệm chi phí server.
- **Quản lý "Visibility Timeout" kém cho tác vụ dài:** Quá trình chạy inference model AI trên ảnh độ phân giải cao có thể mất vài giây đến vài chục giây. Các queue thông thường rất khó quản lý trạng thái "đang xử lý". Điều này dễ dẫn đến lỗi một tấm ảnh bị gửi cho 2 worker cùng xử lý do queue tưởng worker đầu tiên đã treo. JetStream cho phép cấu hình thời gian chờ xác nhận (Ack-Wait) rất linh hoạt phù hợp với các long-running tasks.

C. Giới hạn phạm vi:

- **Độ dài video:** Hỗ trợ xử lý tập hình ảnh, video ngắn có thời lượng từ **15 giây đến 60 giây** nhằm đảm bảo áp lực hợp lý trên cụm server thực nghiệm CPU (không có GPU vật lý).
- **Định dạng hỗ trợ:** Hình ảnh (JPG, PNG, WEBP), Video (MP4 chuẩn H.264).